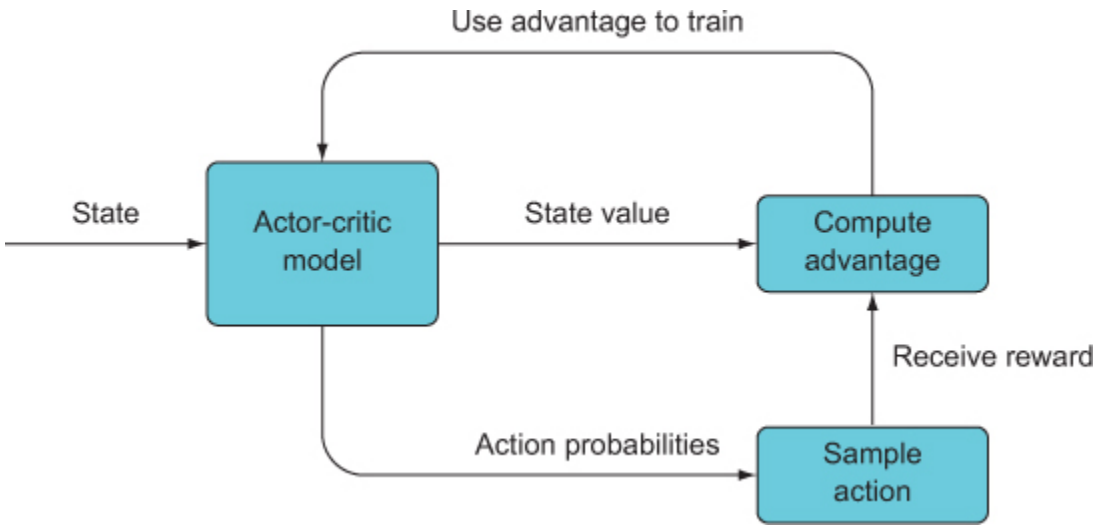
N-step learning is what's in between fully online learning and waiting for a full episode before updating (i.e., Monte Carlo). As the name suggests, we accumulate rewards over *N* steps and then compute our loss and backpropagate. The number of steps can be anywhere from 1, which reduces to fully online learning, to the maximum number of steps in the episode, which is Monte Carlo. Usually we pick something in between to get the advantages of both. We will first show the episodic actor-critic algorithm, and then we will adapt it to *N*-step with *N* set to 10.

Figure 5.10 shows the broad overview of an actor-critic algorithm. An actor-critic model needs to produce both a state value and action probabilities. We use the action probabilities to select an action and receive a reward, which we compare with the state value to compute an advantage. The advantage is ultimately what we use to reinforce the action and train the model.

Figure 5.10. An actor-critic model produces a state value and action probabilities, which are used to compute an advantage value and this is the quantity that is used to train the model rather than raw rewards as with just *Q*-learning.



With that in mind, let's get to coding an actor-critic model to play CartPole. Here's the sequence of steps.

1. Set up our actor-critic model, a two-headed model (or you can set up two independent actor and critic networks). The model accepts a CartPole state as input, which is a vector of 4 real numbers. The actor